

Git: Desvendando o Código do Tempo

"Uma Aventura Interativa no Mundo do
Controle de Versão"

» By Whitebeard Technology





Capítulo 1: Dominando os Fundamentos do Git com PBL

Seção 1.1: Bem-vindo ao Mundo do Git

PBL Início: Imagine que você é um detetive de código, e sua missão é encontrar a melhor maneira de salvar diferentes versões de um projeto sem perder nenhuma informação.

Exemplo: Você começa um projeto de website e faz sua primeira página HTML. Como você pode salvar esse ponto inicial de forma segura?

Exercício Guiado:

1. Crie uma pasta no seu computador para o projeto do website.
2. Abra o terminal ou prompt de comando, navegue até a pasta do projeto e execute `git init`.
3. Crie um arquivo chamado `index.html` e escreva um HTML básico.
4. Adicione o arquivo ao Git com `git add index.html` e faça seu primeiro commit com `git commit -m "Cria página inicial"`.

Seção 1.2: Instalação e Configuração Inicial

PBL Início: Você precisa configurar sua identidade no Git antes de começar a trabalhar em um projeto colaborativo. Como você faria isso?

Exemplo: Você decide contribuir para um projeto open-source e precisa que suas contribuições sejam reconhecidas.

Exercício Guiado:

1. Configure seu nome e email no Git com `git config --global user.name "Seu Nome"` e `git config --global user.email "seuemail@example.com"`.
2. Verifique se a configuração está correta com `git config --list`.

Seção 1.3: Seu Primeiro Repositório

PBL Início: Você quer começar a versionar um projeto de código que já começou. Qual é o primeiro passo?

Exemplo: Você tem um projeto de jogo simples em Python que gostaria de começar a versionar.

Exercício Guiado:

1. Navegue até a pasta do seu projeto de jogo em Python.
2. Execute `git init` para inicializar um repositório Git.
3. Adicione todos os arquivos ao Git com `git add .` e faça um commit com `git commit -m "Inicializa projeto de jogo Python".`

Seção 1.4: Salvando as Mudanças: O Básico dos Commits

PBL Início: Você fez mudanças significativas no seu projeto e quer garantir que elas não sejam perdidas. Como proceder?

Exemplo: Após adicionar uma nova funcionalidade ao seu jogo, você precisa salvar essas mudanças.

Exercício Guiado:

1. Após testar a nova funcionalidade, abra o terminal.
2. Use `git status` para ver as mudanças.
3. Adicione as mudanças ao stage com `git add .` e faça um commit com `git commit -m "Adiciona nova funcionalidade ao jogo".`

Seção 1.5: Branches: Trabalhando em Diferentes Versões Simultaneamente

PBL Início: Você quer experimentar uma nova ideia para seu projeto sem afetar a versão principal. Como você pode fazer isso?

Exemplo: Você decide adicionar um novo nível ao seu jogo, mas não tem certeza se vai mantê-lo.

Exercício Guiado:

1. Crie uma nova branch para o novo nível com `git branch novo-nivel`.
2. Mude para essa branch com `git checkout novo-nivel`.
3. Comece a desenvolver o novo nível.

Seção 1.6: Unindo Tudo com Merge

PBL Início: Após testar e aperfeiçoar a nova funcionalidade na branch secundária, como você a integra na versão principal do seu projeto?

Exemplo: O novo nível do seu jogo foi um sucesso entre os testadores. Agora, é hora de adicioná-lo à versão principal.

Exercício Guiado:

1. Volte para a branch principal com `git checkout main`.
2. Faça o merge da branch `novo-nivel` com `git merge novo-nivel`.
3. Resolva quaisquer conflitos que surgirem.

Seção 1.7: Uh-oh, Conflitos!

PBL Início: Durante o merge, você encontra um conflito. Como você decide qual código manter?

Exemplo: Há diferenças no arquivo de configuração do jogo entre a branch principal e a `novo-nivel`.

Exercício Guiado:

1. Abra o arquivo com conflito.
2. Revise as mudanças conflitantes e decida qual versão manter.
3. Após ajustar o arquivo, faça um novo commit com `git add .` e `git commit -m "Resolve conflito de merge"`.

Desafio Final: O Grande Merge

Agora que você aprendeu sobre commits, branches e merges, seu desafio é criar um projeto do zero, adicionar uma nova funcionalidade em uma branch separada, testá-la e, finalmente, fazer um merge dessa funcionalidade de volta à branch principal. Ao longo do caminho, você encontrará um conflito proposital que deverá resolver.

Capítulo 2: Resolvendo o Desafio Final - Git em Ação

Introdução ao Desafio

Bem-vindo ao desafio final: "Git: A Conquista do Controle de Versão". Aqui, vamos aplicar tudo o que aprendemos sobre Git para criar, modificar e resolver conflitos em um projeto. Prepare-se para colocar suas habilidades à prova!

Etapa 1: Iniciando o Projeto

1. **Configuração Inicial:** Abra o terminal, navegue até a pasta desejada e execute `git init`.
2. **Primeiro Commit:** Crie um arquivo `README.md`, adicione conteúdo básico e faça seu primeiro commit.

Etapa 2: Desenvolvendo uma Nova Funcionalidade

1. **Branch de Funcionalidade:** Crie uma branch `git branch nova-funcionalidade` e mude para ela com `git checkout nova-funcionalidade`.
2. **Adicionando Funcionalidade:** Faça alterações ou adicione novos arquivos. Após as mudanças, faça um commit.

Etapa 3: Gerando um Conflito

1. **Preparando o Conflito:** Volte para a branch principal (`git checkout main`) e altere o mesmo arquivo modificado na branch de funcionalidade.
2. **Commit das Mudanças:** Faça commit dessas alterações na branch principal.

Etapa 4: Merge e Resolução de Conflitos

1. **Tentativa de Merge:** Execute `git merge nova-funcionalidade`. O Git indicará um conflito.
2. **Resolvendo Conflitos:** Abra o arquivo conflitante, resolva as diferenças manualmente.
3. **Finalizando o Merge:** Após resolver, faça `git add .` e `git commit -m "Resolve conflito"`.

Conclusão

Você acabou de realizar um ciclo completo de desenvolvimento com o Git: criação de projeto, desenvolvimento em branches, resolução de conflitos e merge. Parabéns! Agora, você possui um conhecimento prático robusto sobre como o Git funciona em cenários reais de desenvolvimento de software.